AI for Merger Negotiation

Business decision modeling under negotiation constraints

Daxton Huynh | Technical case study

Executive perspective

A price negotiation is a sequence of decisions: propose a price, accept an offer, continue bargaining, or walk away. Each choice changes the options available later. This project starts with a business question: how can a decision process represent concession limits, timing and competing interests while remaining computationally manageable?

The study builds a transparent simulation of bilateral merger bargaining. Exact search provides a reference decision rule; pruning and limited look-ahead examine how much computation can be saved and whether resulting negotiations change. All scenarios are synthetic. The intended value is a controlled framework for understanding decision rules, rather than a calibrated estimate of actual transaction outcomes.

Business question	Finding in this experiment
Can the same modeled decision quality require less search?	Ordered Alpha-Beta reduces mean node visits by 84.6% while matching exact-search mean utility and final-status profile.
Does deeper evaluation change outcomes?	Depth-3 heuristics are closer to the exact baseline than depth-2 heuristics in realized utility.
Does adding more business detail improve the heuristic?	At matched depth, h1/h2/h3 have identical aggregate utility and agreement rates in this batch. No outcome advantage for h3 is established.

How to use this report

Read the business framing and worked example first, then inspect the model and benchmark evidence. The final sections translate findings into design implications, validation boundaries and a practical development roadmap. Detailed code and CSV outputs accompany the report.

1. Business problem and research questions

Decision context

A buyer has a maximum economically feasible price, including modeled synergy; a seller has a minimum acceptable price. Even with an overlapping price range, the parties may prefer different prices or terminate negotiations. Available concessions and elapsed time constrain subsequent moves. A useful model must make those constraints explicit before an algorithm chooses a move.

Potential users of a simulation of this kind include analysts exploring bargaining assumptions and teams comparing automated decision policies. The present project demonstrates the modeling approach; it does not establish adoption, commercial demand or financial benefit for these users.

Business concept	Representation	What it helps examine
Walk-away price	Buyer reservation plus synergy; seller reservation	Which prices are individually feasible
Negotiating flexibility	Remaining concession budgets	How earlier concessions constrain later choices
Cost of delay	Cost per completed round	How continuation changes the modeled score
Negotiation deadline	Finite maximum rounds	When a proposed deal can expire
Decision effort	Search depth and node visits	What is gained or lost by limiting look-ahead

Research questions

RQ1. Can exact-search decisions be obtained with fewer explored states?
RQ2. How do search depth and heuristic design change realized bargaining outcomes?
RQ3. Does ordering candidate actions improve computational efficiency consistently?

Success criteria

For exact pruning, preserve the computed value and reduce node visits. For approximate search, report computational savings alongside outcome deviation, agreement rate and price. A higher buyer score in self-play is not sufficient evidence of a better policy against a strong opponent. No claim of real financial return is made.

2. Negotiation model and business assumptions

Symbol	Meaning
B, S, Y	Buyer reservation, seller reservation, synergy
H = B + Y; L = S	Upper and lower feasible price bounds
c, r, R	Delay cost per round, current round, maximum round
Cb, Cs	Buyer and seller remaining concession budgets
b, a, q	Last buyer bid, last seller ask, current offer
P	Breakdown penalty in relative score units

The model is deterministic and fully observable. Buyer is the maximizing player; seller minimizes the same buyer-facing score. Both parties know the model parameters. Prices lie on a discrete action grid. Only overlapping feasible zones (L less than or equal to H) are supported. Numbers are abstract price/score units, not CAD valuations.

Actions and constraints

At each turn, a party may accept a feasible current offer, exit, or propose a permitted price. Buyer offers cannot decrease and seller offers cannot increase. The initial buyer anchor is L; the initial seller anchor is H. Remaining budget limits incremental movement from the previous own offer.

```
Buyer range: [b, min(b + Cb, H)]
Seller range: [max(a - Cs, L), a]
```

Before a party has made an offer, substitute its initial anchor for b or a. After an offer, deduct only the incremental concession from that party's budget. The grid is enumerated from the lower bound in offer_step increments; an off-grid upper endpoint is not necessarily included.

Timing and information

The round starts at 1 and increments after a seller counteroffer. When r exceeds R, the state is terminal: a final-boundary counteroffer can expire before a response. Acceptance and exit terminate immediately. This convention is part of the simulation and would need redesign for a business protocol that guarantees a response to the final offer.

Full information, a single price issue and deterministic synergy simplify analysis but omit due diligence, financing, contractual contingencies and private valuation information. The scenario labels describe constructed parameter groups; they do not prove which party has greater bargaining power in reality.

3. Worked business example

Consider scenario BA1: B=100, S=70, synergy=10, buyer budget=20, seller budget=30, delay cost=0, and three rounds. The feasible price zone is 70 to 110. The following is a legal illustrative path chosen to explain the mechanics; it is not presented as the algorithm's optimal trajectory.

Step	Action	Business interpretation	Budget after move
1	Buyer offers 80	Buyer moves 10 above its initial anchor of 70.	Buyer 10; seller 30
2	Seller offers 100	Seller moves 10 below its initial anchor of 110.	Buyer 10; seller 20
3	Buyer offers 90	Buyer spends its remaining 10 units of flexibility.	Buyer 0; seller 20
4	Seller accepts 90	Price is above the seller minimum of 70.	Negotiation ends

What the score means

```
Buyer gain = 110 - 90 = 20
Seller gain = 90 - 70 = 20
Relative utility = 20 - 20 - 0 = 0
```

A utility of zero here means equal modeled gains after the delay adjustment. It does not mean the deal has zero economic value. At price 100, the buyer gain would be 10 and seller gain 30, giving a relative score of -20. A negative relative score can therefore coexist with a feasible agreement that benefits both parties.

Why feasibility does not guarantee agreement

The model optimizes a relative adversarial score. Acceptance, exit and horizon outcomes are compared on that scale. A party can reject a feasible agreement because its alternative has a more attractive modeled score. Agreement rate must therefore be interpreted together with the payoff convention, not as an empirical estimate of deal completion probability.

A testable modeling requirement

After buyer offers 80, the remaining budget of 10 must allow a later bid of 90. Likewise, seller's remaining budget must be applied to its last ask. Regression tests explicitly verify these multi-step paths so that the business meaning of incremental concessions is preserved.

4. Objective, search and heuristic design

```
D(r) = (min(r, R + 1) - 1) * c
U(accepted at p) = (H - p) - (p - L) - D(r)
U(buyer exit) = -P - D(r)
U(seller exit) = +P - D(r)
U(horizon) = -P / 2 - D(r)
```

These are the default asymmetric_relative payoffs. A seller exit can score positively for buyer, which is a modeling convention rather than a cash gain. The optional symmetric_breakdown mode sets every failure to -P-D(r); it still does not create independent economic utility functions for the two parties.

Search as a decision mechanism

Minimax evaluates legal continuations to terminal states, choosing the maximum score on buyer turns and minimum on seller turns. Exact Alpha-Beta skips branches that cannot improve the current bound, preserving the exact value. Depth-limited search stops after a specified number of actions, not complete bargaining rounds, and substitutes a heuristic for terminal utility. Terminal states are always evaluated first.

```
h1(s) = H-L if no current offer
= q-L if q < L
= H-q otherwise
h2(s) = h1(s) - (r-1)*c
```

The last h1 branch also handles offers above H by returning a negative value. Under the legal action generator, offers lie within the feasible zone; out-of-zone branches are defensive definitions. h1 uses buyer price attractiveness and omits seller gain, so its scale differs from terminal utility.

```
g = max(0, a-b); T = max(0, R-r+1)
risk = P*I(g > Cb+Cs) + (P/2)*I(T <= 1 and g > 0)
h3(s) = h2(s) - risk
```

I(condition) is 1 when true and 0 otherwise. Initial anchors fill missing b/a. The bid-ask gap measures unresolved aspirations. Its penalty is a deterministic pressure proxy, not a probability and not a proof that agreement is impossible: either party may accept the opponent's feasible offer. This distinction is essential to interpreting h3.

5. Experimental design

ID	Group	B	S	Y	c	Cb	Cs
BA1	buyer	100	70	10	0	20	30
BA2	buyer	100	80	20	0	20	30
BA3	buyer	110	85	15	0	20	30
BL1	balanced	105	85	10	0	20	20
BL2	balanced	110	90	10	0	20	20
BL3	balanced	115	95	10	0	20	20
SA1	seller	110	90	5	3	15	25
SA2	seller	115	95	5	3	15	25
SA3	seller	120	100	5	4	15	25

All scenarios use a maximum of 3 rounds, offer step 5 and breakdown penalty 20. The fixed scenarios are reproducible; no random seed is required. Group comparisons combine several parameter changes and cannot isolate causal effects of a single business variable.

Comparison	Configurations	Purpose
Exact search	Minimax; Alpha-Beta ordered/unordered	Value preservation and pruning effort
Matched heuristic design	h1, h2, h3 each at depth 2 and depth 3	Separate heuristic effects from depth effects
Approximate ordering	h3 depth 3 ordered/unordered	Explore ordering under cutoff evaluation

Both parties use the selected method throughout a negotiation (self-play). There are ten configurations across nine scenarios, producing 90 run records. The ordered Minimax run provides the per-scenario baseline. Timing is measured once per configuration/scenario, without warm-up or repeated trials, and is illustrative only.

Move ordering

Ordered search visits acceptance first, offers second and exit last. Buyer offer candidates are sorted from higher to lower price; seller offers from lower to higher price. This is an implemented search-order rule, not a demonstrated ranking of business attractiveness. Strict improvement determines the selected move, so equal-value ties can yield different actions when ordering changes.

Measurement definitions

Nodes count recursive visits, including terminal/cutoff calls, accumulated over successive decisions. Utility and final status describe the realized self-play path. Mean final price excludes no-deal records. Absolute utility gap is the average of scenario-level absolute differences from Minimax; it is not regret or exploitability. Reduction in mean node counts and mean per-scenario percentage reductions are different statistics.

6. Results and business interpretation

Method	Nodes	Utility	Deal rate	Abs. gap
Alpha-Beta	789.2	-13.111	66.7%	0.000
Alpha-Beta (unordered)	1,258.4	-13.111	66.7%	0.000
HAB h1 (d=2)	59.7	1.667	100.0%	14.778
HAB h1 (d=3)	433.7	-13.333	66.7%	0.222
HAB h2 (d=2)	59.7	1.667	100.0%	14.778
HAB h2 (d=3)	437.0	-13.333	66.7%	0.222
HAB h3 (d=2)	59.7	1.667	100.0%	14.778
HAB h3 (d=3)	441.3	-13.333	66.7%	0.222
HAB h3 (d=3, unordered)	359.0	-13.333	66.7%	0.222
Minimax	5,127.3	-13.111	66.7%	0.000

RQ1: equivalent exact values with less search

Ordered Alpha-Beta reduces mean node visits by 84.6% relative to Minimax. It achieves the same aggregate realized utility and final-status profile. This supports pruning as an efficiency improvement within this model. Fewer node visits do not translate directly into a measured business cost saving.

RQ2: depth matters; heuristic superiority is not established

All depth-2 variants produce mean utility 1.667, 100% agreements and absolute utility gap 14.778. At depth 3, all three produce -13.333, 66.7% agreements and gap 0.222. The deeper configuration is closer to exact self-play outcomes, but h3 does not outperform h1/h2 on these outcome measures. Identical aggregates alone do not establish identical policies in every state.

The higher buyer score at depth 2 must not be labeled better strategy: seller is approximate too. The experiment changes both decision makers simultaneously. A fixed exact opponent and best-response analysis would be required to assess strategic robustness.

RQ3: ordering is method-dependent

Exact ordered Alpha-Beta visits fewer nodes than its unordered variant (789.2 versus 1,258.4). For h3 at depth 3, unordered visits fewer nodes (359.0 versus 441.3) while matching aggregate utility. Ordering is therefore an empirical design choice for this simulator, not a universal performance guarantee.

7. Visual comparison and scenario context

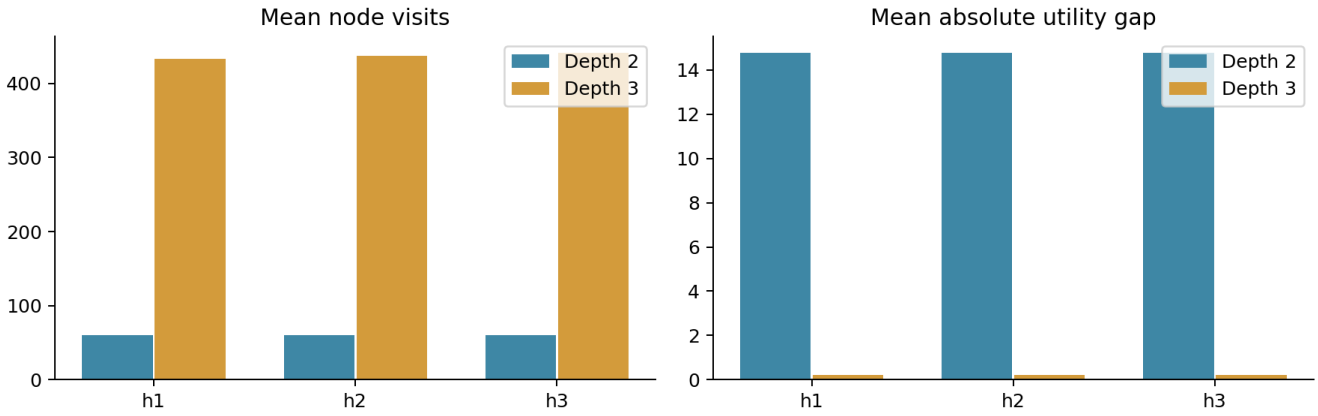


Figure 1. Matched-depth comparison. More search at depth 3 coincides with smaller deviation from the exact self-play baseline; the three heuristic outcomes remain equal in aggregate.

Group	Method	Utility	Agreement
balanced	HAB h1 (d=2)	0.000	100.0%
balanced	HAB h3 (d=3)	-10.000	100.0%
balanced	Minimax	-10.000	100.0%
buyer advantaged	HAB h1 (d=2)	0.000	100.0%
buyer advantaged	HAB h3 (d=3)	-10.000	100.0%
buyer advantaged	Minimax	-10.000	100.0%
seller advantaged	HAB h1 (d=2)	5.000	100.0%
seller advantaged	HAB h3 (d=3)	-20.000	0.0%
seller advantaged	Minimax	-19.333	0.0%

Each group contains only three scenarios. Apparent differences should motivate sensitivity experiments, not general claims about buyers or sellers. Delay cost, reservation values, synergy and flexibility vary together across the constructed groups.

8. Validation, implications and next steps

Validation performed	Scope
Exact-search agreement	Minimax and Alpha-Beta values at nine roots and immediate successors, with ordering on/off
Concession regressions	Buyer and seller retain legal use of remaining budget after earlier concessions
Input/action checks	Invalid step rejected; illegal acceptance rejected
Heuristic activation	A constructed late-round state activates h3 pressure
Execution pipeline	Batch output and plotting completed; PDF tables built from output CSVs

Six automated test methods passed in the local environment. These checks cover sampled states and specific regressions, not exhaustive correctness for all parameter combinations. CI configuration runs the tests and batch script on Python 3.10 and 3.12; remote CI results must be checked after publication.

Business implications

Make constraints explicit. A small error in how remaining flexibility is modeled changes the options the algorithm considers.
Evaluate multiple objectives. Agreement rate, relative score, price and computational effort answer different questions.
Validate additional complexity. More business factors in a heuristic do not automatically produce better decisions.
Separate simulation insight from field impact. Real negotiation support would require stakeholder-defined payoffs, credible data and prospective evaluation.

Limits and development priorities

First, compare approximate agents against a fixed exact opponent. Second, vary one business parameter at a time and include broader horizons. Third, repeat runtime trials and report dispersion. Fourth, test alternative payoff conventions and independent utilities for both sides. Finally, introduce private information and multiple contractual issues before considering operational use. Non-overlapping feasible zones currently raise an error and are outside the experiment.

Conclusion

This project demonstrates how a business decision can be translated into explicit state, constraints, alternatives and evaluation criteria. Exact pruning substantially reduces search effort in the tested environment. Depth changes approximate outcomes, while the current scenarios do not establish an advantage for a more elaborate heuristic. The strongest contribution is a transparent, testable decision-modeling workflow and a clearly bounded set of computational findings.

9. Reproduction and evidence guide

Repository component	Role
merger_negotiation_game.py	State, constraints, utility, heuristics and search
merger_negotiation_batch_experiments.py	Scenario/method definitions and run-level measurement
plot_negotiation_results.py	Benchmark figures and summary tables
outputs/negotiation_batch_raw_results.csv	Ninety detailed run records
outputs/negotiation_batch_summary_by_method.csv	Aggregate results used in this report
outputs/negotiation_batch_summary_by_class_method.csv	Scenario-group comparisons
tests/test_search.py	Six validation methods
scripts/build_report.py	PDF and matched-depth figure generation

Run from the repository root

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-report.txt
python merger_negotiation_batch_experiments.py
python plot_negotiation_results.py
python -m unittest discover -s tests -v
python scripts/build_report.py
```

On Windows PowerShell, activate with .venv\Scripts\Activate.ps1. Keep the three root Python modules together because their imports and output locations assume that layout. requirements.txt contains plotting dependencies; requirements-report.txt adds the PDF library. Runtime varies across machines and runs. Deterministic outcome fields should reproduce under the same code and parameters.

Evidence provenance

Numerical claims are derived from the included CSV outputs, and model equations are transcribed from the implementation. The example in Section 3 is an illustrative legal path, not a measured optimal policy. This report does not claim empirical validation, literature novelty or commercial deployment. Implementation changes are recorded separately in CHANGELOG.md.